

TradeTracker

Technická dokumentace projektu

Předmět: Objektovy orientované programování / Pavlo Koryk IT2A

Jazyk: TypeScript

Fáze: I, II, III – kompletní projekt

GitHub: <https://github.com/pupiko-co/TradeTracker>

<https://github.com/pupiko-co/TradeTracker>

Obsah

1. Cíle projektu a popis aplikace
2. Architektonický návrh a UML diagram
3. Popis tříd a OOP principy
4. Propojení logiky s HTML (DOM)
5. Uživatelská příručka
6. Výpis zdrojového kódu

1. Cíle projektu a popis aplikace

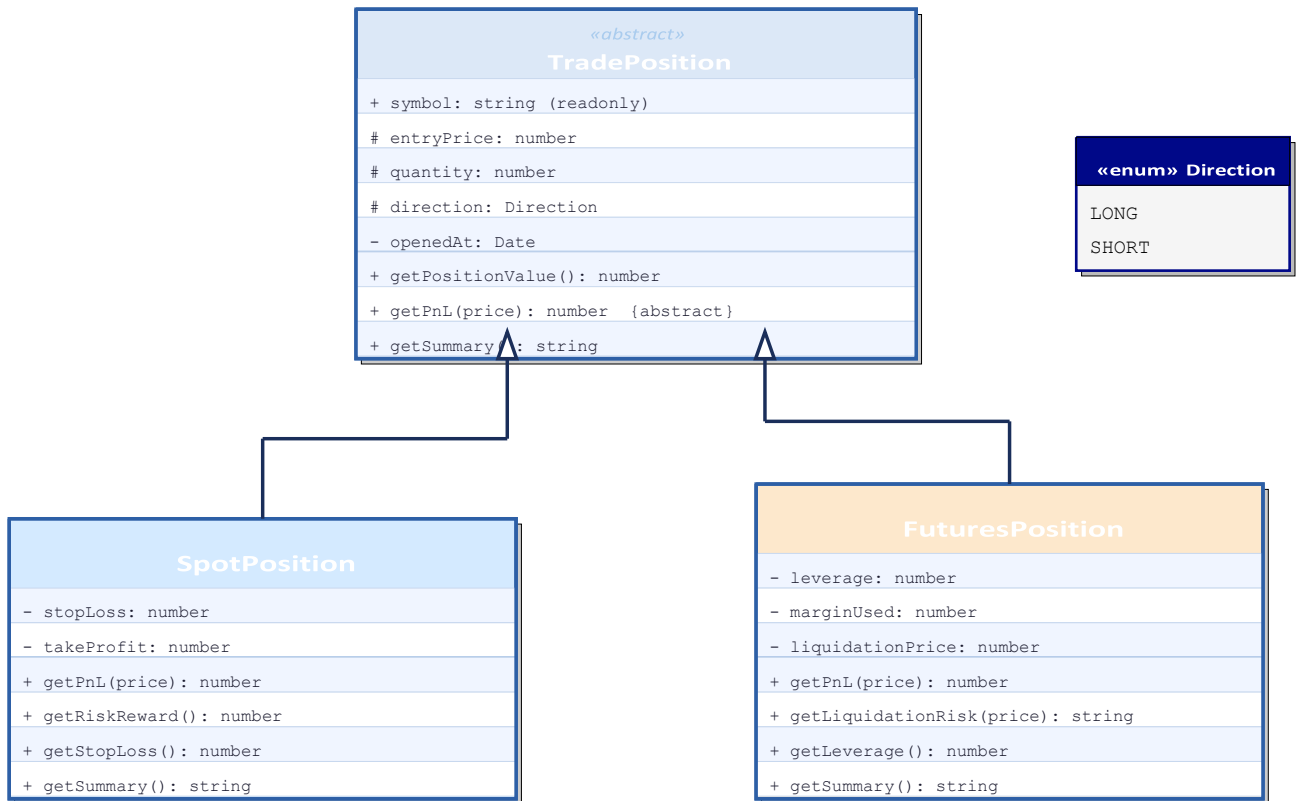
TradeTracker je webová aplikace napsaná v TypeScriptu, která umožňuje obchodníkům na finančních trzích správu a analýzu jejich obchodních pozic. Podporuje dva typy pozic – **spotové** a **futures** – a v reálném čase přepočítává zisk/ztrátu (P&L;) na základě zadané aktuální ceny.

Co aplikace dělá:

- Eviduje pozice dvou typů: **Spot** (přímý nákup) a **Futures** (pákový obchod s marginí).
- Počítá **nerealizovaný P&L;** pro každou pozici.
- U spotových pozic zobrazuje **Risk:Reward ratio**.
- U futures zobrazuje **páku, marži a riziko likvidace**.
- Zobrazuje **souhrnný report portfolio** s celkovým P&L;
- Validuje vstupní data – brání vytvoření neplatné pozice.

2. Architektonický návrh a UML diagram

Aplikace je postavena na hierarchii tříd. Abstraktní třída **TradePosition** definuje společné vlastnosti a předpisuje abstraktní metodu `getPnL()`. Obě konkrétní třídy ji implementují vlastní logikou.



Obr. 1 – UML diagram tříd. Prázdná šipka = dědičnost. + public, - private, # protected. «abstract» = abstraktní třída.

3. Popis tříd a OOP principy

TradePosition – abstraktní bazová třída

Zapouzdřuje vlastnosti společné pro všechny pozice. Používá modifikátory: public readonly pro symbol, protected pro cenu a objem, private pro datum. Abstraktní getPnL() nutí potomky implementovat vlastní výpočet.

SpotPosition – spotové pozice

Rozšiřuje bázi o stopLoss a takeProfit. $P\&L; = (cena - vstup) \times objem \times směr$.
Přidává getRiskReward() pro výpočet poměru rizika.

FuturesPosition – futures pozice

Rozšiřuje bázi o páku, marži a likvidaci. $P\&L; = (cena - vstup) \times objem \times páka \times směr$.
getLiquidationRisk() vrátí LOW / MEDIUM / HIGH.

Polymorfismus

Portfolio je TradePosition[] s mixem oběma typy. Volání getPnL() a getSummary() automaticky vybere správnou implementaci podle skutečného typu – volající kód typ nezná. To je polymorfismus.

4. Propojení logiky s HTML (DOM)

Soubor `app.ts` tvoří most mezi OOP třídami a HTML. Webpack sestaví všechny soubory do `dist/bundle.js` který je načten v `index.html`.

Postup při přidání pozice:

- Uživatel vyplní formulář a klikne **Přidat pozici**.
- `app.ts` načte hodnoty z HTML inputů přes `getElementById()`.
- Tovární funkce `createPosition()` vytvoří instanci `SpotPosition` nebo `FuturesPosition`.
- Objekt je uložen do pole `portfolio[]`.
- `renderTable()` překreslí tabulku – volá `getPnL()` polymorfně.
- `updateSummary()` aktualizuje souhrné karty.

Responzivní design

Na mobilech se tabulka přepne do blokového zobrazení pomocí `@media` (`max-width: 600px`).

Bučky zobrazují název přes atribut `data-label`.

5. Uživatelská příručka

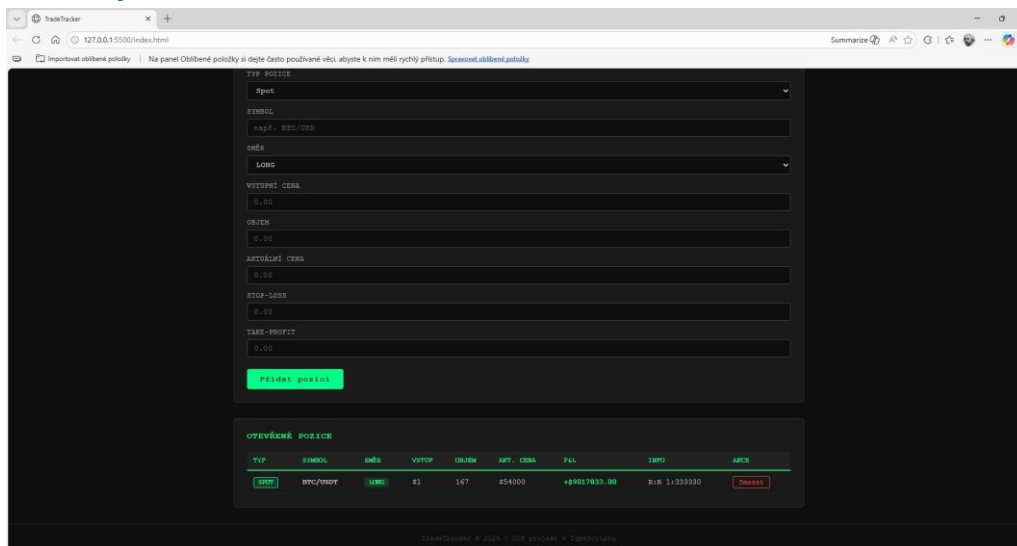
Spuštění aplikace

- git clone <https://github.com/pupiko-co/TradeTracker.git>
- npm install
- npm run build
- Otevřete index.html v prohlížeči.

Popis rozhraní

- **Souhrnné karty** – počet pozic, celková hodnota, celkový P&L;
- **Formulář** – výběr Spot/Futures, symbol, směr, ceny a parametry.
- **Přidat pozici** – přidá pozici. Při chybě zobrazí hlášku.
- **Tabulka** – všechny pozice s P&L; (zelená = zisk, červená = ztráta).
- **Smazat** – odstraňuje pozici z portfolia.

Screenshot aplikace



Obr. 2 – Hlavní rozhraní TradeTracker v prohlížeči. Formulář pro přidání pozice a tabulka se spotovou pozicí BTC/USDT.

6. Výpis zdrojového kódu

Klíčové části zdrojového kódu projektu. Úplný kód je na GitHubu.

TradePosition.ts – abstraktní bázeová třída

```
export enum Direction { LONG = "LONG", SHORT = "SHORT" }

export abstract class TradePosition {    public readonly
symbol: string;           // verejne, nemenne    protected
entryPrice: number;       // dostupne potomkum    protected
quantity: number;    protected direction: Direction;
private openedAt: Date;           // pouze tato trida

    constructor(symbol, entryPrice, quantity, direction) {
if (!symbol || symbol.trim() === "")        throw new
Error("Symbol nesmi byt prazdny.");        if (entryPrice
<= 0)        throw new Error("Vstupni cena musi byt
kladna.");        if (quantity <= 0)        throw new
Error("Objem musi byt kladny.");        this.symbol =
symbol.trim().toUpperCase();        this.entryPrice =
entryPrice;        this.quantity = quantity;
this.direction = direction;        this.openedAt = new
Date();    }

    public getPositionValue(): number { return this.entryPrice * this.quantity; }
public abstract getPnL(currentPrice: number): number; // abstraktni    public
getSummary(): string {        return '['+this.symbol+']' +this.direction
+' | Vstup: '+this.entryPrice+' | Objem: '+this.quantity;
    }
}
```

SpotPosition.ts – spotové pozice

```
export class SpotPosition extends TradePosition {
private stopLoss: number;    private takeProfit:
number;

    constructor(symbol, entryPrice, quantity, direction, stopLoss, takeProfit) {
super(symbol, entryPrice, quantity, direction); // vola rodice    if
(stopLoss <= 0) throw new Error("Stop-loss musi byt kladny.");    if
(takeProfit <= 0) throw new Error("Take-profit musi byt kladny.");    //
validace SL/TP vuci smeru obchodu...
        this.stopLoss = stopLoss;
this.takeProfit = takeProfit;    }

    // Vlastni implementace getPnL pro spot    // Vzorec: (aktualni
cena - vstup) x objem x smer    public getPnL(currentPrice: number):
number {        const mult = this.direction === Direction.LONG ? 1 : -
1;        return (currentPrice - this.entryPrice) * this.quantity *
mult;    }

    // Pomer riziko:odmena    public getRiskReward(): number {        const
risk    = Math.abs(this.entryPrice - this.stopLoss);        const reward =
Math.abs(this.takeProfit - this.entryPrice);        return risk === 0 ?
Infinity : parseFloat((reward/risk).toFixed(2));
}
```



```
}

    public override getSummary(): string {        return
super.getSummary()          + ' | SL: '+this.stopLoss+' |
TP: '+this.takeProfit
        + ' | R:R = 1:'+this.getRiskReward();
    }
}
```

FuturesPosition.ts – futures pozice

```
export class FuturesPosition extends TradePosition {
  private leverage: number;   private marginUsed:
  number;   private liquidationPrice: number;

  constructor(symbol, entryPrice, quantity, direction, leverage, marginUsed) {
    super(symbol, entryPrice, quantity, direction);   if (leverage < 1)   throw
    new Error("Paka musi byt min. 1x.");   if (marginUsed <= 0) throw new
    Error("Marze musi byt kladna.");   this.leverage   = leverage;
    this.marginUsed = marginUsed;   // Likvidacni cena: vstup * (1 +/- 1/paka *
    0.9)   const f = (1 / leverage) * 0.9;   this.liquidationPrice = direction
    === Direction.LONG   ? parseFloat((entryPrice * (1-f)).toFixed(2))
    : parseFloat((entryPrice * (1+f)).toFixed(2));
  }

  // P&L nasobeny pakou: (cena - vstup) x objem x paka x smer   public
  getPnL(currentPrice: number): number {   const mult = this.direction ===
  Direction.LONG ? 1 : -1;   return (currentPrice-
  this.entryPrice)*this.quantity*this.leverage*mult;   }

  // Riziko likvidace (LOW / MEDIUM / HIGH)   public
  getLiquidationRisk(currentPrice: number): string {   const pct =
  Math.abs(currentPrice-this.liquidationPrice)/currentPrice*100;   if (pct >
  15) return "LOW";   if (pct > 5)   return "MEDIUM";   return "HIGH";   }

  public override getSummary(): string {
    return super.getSummary()   +' |
    Paka: '+this.leverage+'x'
    +' | Marze: $'+this.marginUsed
    +' | Likvidace: '+this.liquidationPrice;
  }
}
```

app.ts – propojení s DOM

```
// portfolio = pole { position: TradePosition, currentPrice: number }
let portfolio: { position: TradePosition; currentPrice: number }[] = [];

// Tovární funkce - vytvori spravny typ podle "type" function
createPosition(data: PositionData): TradePosition {   if (data.type
=== "spot")   return new SpotPosition(data.symbol, data.entryPrice,
data.quantity, data.direction, data.stopLoss, data.takeProfit);
else   return new FuturesPosition(data.symbol, data.entryPrice,
data.quantity, data.direction, data.leverage, data.marginUsed); }

// Polymorfni pruchod portfolioem function renderTable(): void {   for
(const item of portfolio) {   const pnl =
item.position.getPnL(item.currentPrice); // polymorfni   const sum =
item.position.getSummary();   // polymorfni   // ...
vykresli do HTML tabulky ...
  }
}

// Pridani pozice pres formular document.getElementById("add-
btn").addEventListener("click", () => {   const type   =
(document.getElementById("type") as HTMLSelectElement).value;   const symbol =
(document.getElementById("symbol") as HTMLInputElement).value;   // ... nacteni
dalsich poli ...   try {   const position = createPosition(formData);
portfolio.push({ position, currentPrice });   renderTable();   // prekresli
DOM   updateSummary(); // aktualizuje karty   } catch(e) {
errorEl.textContent = (e as Error).message; // zobrazi chybu   }
});
```